CHAPTER 6

Measuring and Governing Architecture Characteristics

Architects must deal with the extraordinarily wide variety of architecture characteristics across all different aspects of software projects. Operational aspects like performance, elasticity, and scalability comingle with structural concerns such as modularity and deployability. This chapter focuses on concretely defining some of the more common architecture characteristics and building governance mechanisms for them.

Measuring Architecture Characteristics

Several common problems exist around the definition of architecture characteristics in organizations:

They aren't physics
: Many architecture characteristics in common usage have vague meanings. For example, how does an architect design for *agility* or *deployability*? The industry has wildly differing perspectives on common terms, sometimes driven by legitimate differing contexts, and sometimes accidental.

Wildly varying definitions
: Even within the same organization, different departments may disagree on the definition of critical features such as *performance*. Until developers, architecture, and operations can unify on a common definition, a proper conversation is difficult.

Too composite

Many desirable architecture characteristics comprise many others at a smaller scale. For example, developers can decompose agility into characteristics such as modularity, deployability, and testability.

Objective definitions for architecture characteristics solve all three problems: by agreeing organization-wide on concrete definitions for architecture characteristics, teams create a ubiquitous language around architecture. Also, by encouraging objective definitions, teams can unpack composite characteristics to uncover measurable features they can objectively define.

Operational Measures

Many architecture characteristics have obvious direct measurements, such as performance or scalability. However, even these offer many nuanced interpretations, depending on the team's goals. For example, perhaps a team measures the average response time for certain requests, a good example of an operational architecture characteristics measure. But if teams only measure the average, what happens if some boundary condition causes 1% of requests to take 10 times longer than others? If the site has enough traffic, the outliers may not even show up. Therefore, a team may also want to measure the maximum response times to catch outliers.

The Many Flavors of Performance

Many of the architecture characteristics we describe have multiple, nuanced definitions. Performance is a great example. Many projects look at general performance: for example, how long request and response cycles take for a web application. However, architects and DevOps engineers have performed a tremendous amount of work on establishing performance budgets: specific budgets for specific parts of the application. For example, many organizations have researched user behavior and determined that the optimum time for first-page render (the first visible sign of progress for a webpage, in a browser or mobile device) is 500 ms—half a second; Most applications fall in the double-digit range for this metric. But, for modern sites that attempt to capture as many users as possible, this is an important metric to track, and the organizations behind them have built extremely nuanced measures.

Some of these metrics have additional implications for the design of applications. Many forward-thinking organizations place *K-weight budgets* for page downloads: a maximum number of bytes' worth of libraries and frameworks allowed on a particular page. Their rationale behind this structure derives from physics constraints: only so many bytes can travel over a network at a time, especially for mobile devices in high-latency areas.

High-level teams don't just establish hard performance numbers; they base their definitions on statistical analysis. For example, say a video streaming service wants to monitor scalability. Rather than set an arbitrary number as the goal, engineers measure the scale over time and build statistical models, then raise alarms if the real-time metrics fall outside the prediction models. A failure can mean two things: the model is incorrect (which teams like to know) or something is amiss (which teams also like to know).

The kinds of characteristics that teams can now measure are evolving rapidly, in conjunction with tools and nuanced understanding. For example, many teams recently focused on performance budgets for metrics such as *first contentful paint* and *first CPU idle*, both of which speak volumes about performance issues for users of webpages on mobile devices. As devices, targets, capabilities, and myriad other things change, teams will find new things and ways to measure.

Structural Measures

Some objective measures are not so obvious as performance. What about internal structural characteristics, such as well-defined modularity? Unfortunately, comprehensive metrics for internal code quality don't yet exist. However, some metrics and common tools do allow architects to address some critical aspects of code structure, albeit along narrow dimensions.

An obvious measurable aspect of code is complexity, defined by the *cyclomatic complexity* metric.

Cyclomatic Complexity

Cyclomatic Complexity (CC) is a code-level metric designed to provide an object measure for the complexity of code, at the function/method, class, or application level, developed by Thomas McCabe, Sr., in 1976.

It is computed by applying graph theory to code, specifically decision points, which cause different execution paths. For example, if a function has no decision statements (such as `if` statements), then `CC = 1`. If the function had a single conditional, then `CC = 2` because two possible execution paths exist.

The formula for calculating the CC for a single function or method is $CC = E - N + 2$, where `N` represents *nodes* (lines of code), and `E` represents *edges* (possible decisions). Consider the C-like code shown in Example 6-1.

Example 6-1. Sample code for cyclomatic complexity evaluation

```
public void decision(int c1, int c2) {
    if (c1 < 100)
        return 0;
    else if (c1 + C2 > 500)
       return 1;
    else
      return -1;
}
```

The cyclomatic complexity for Example 6-1 is 3 (=3 – 2 + 2); the graph appears in Figure 6-1.

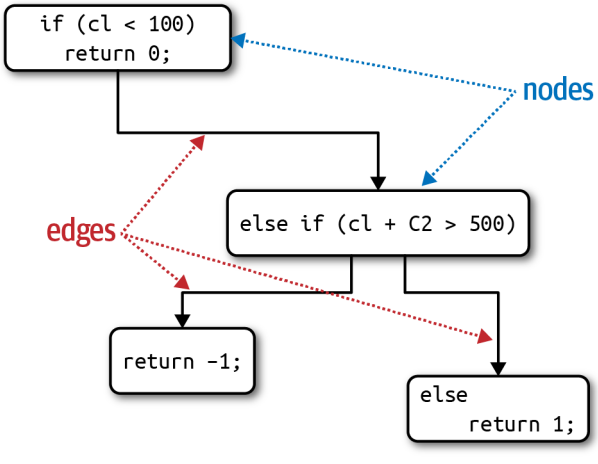

Figure 6-1. Cyclomatic Complexity for the decision function

The number 2 appearing in the cyclomatic complexity formula represents a simplification for a single function/method. For fan-out calls to other methods (known as *connected components* in graph theory), the more general formula is $CC = E - N + 2P$, where P represents the number of connected components.

Architects and developers universally agree that overly complex code represents a code smell; it harms virtually every one of the desirable characteristics of code bases: modularity, testability, deployability, and so on. Yet if teams don't keep an eye on gradually growing complexity, that complexity will dominate the code base.

What's a Good Value for Cyclomatic Complexity?

A common question the authors receive when talking about this subject is: what's a good threshold value for CC? Of course, like all answers in software architecture: it depends! It depends on the complexity of the problem domain. For example, if you have an algorithmically complex problem, the solution will yield complex functions. Some of the key aspects of CC for architects to monitor: are functions complex because of the problem domain or because of poor coding? Alternatively, is the code partitioned poorly? In other words, could a large method be broken down into smaller, logical chunks, distributing the work (and complexity) into more well-factored methods?

In general, the industry thresholds for CC suggest that a value under 10 is acceptable, barring other considerations such as complex domains. We consider that threshold very high and would prefer code to fall under five, indicating cohesive, well-factored code. A metrics tool in the Java world, Crap4J, attempts to determine how poor (crappy) your code is by evaluating a combination of CC and code coverage; if CC grows to over 50, no amount of code coverage rescues that code from crappiness. The most terrifying professional artifact Neal ever encountered was a single C function that served as the heart of a commercial software package whose CC was over 800! It was a single function with over 4,000 lines of code, including the liberal use of `GOTO` statements (to escape impossibly deeply nested loops).

Engineering practices like test-driven development have the accidental (but positive) side effect of generating smaller, less complex methods on average for a given problem domain. When practicing TDD, developers try to write a simple test, then write the smallest amount of code to pass the test. This focus on discrete behavior and good test boundaries encourages well-factored, highly cohesive methods that exhibit low CC.

Process Measures

Some architecture characteristics intersect with software development processes. For example, agility often appears as a desirable feature. However, it is a composite architecture characteristic that architects may decompose into features such as testability, and deployability.

Testability is measurable through code coverage tools for virtually all platforms that assess the completeness of testing. Like all software checks, it cannot replace thinking and intent. For example, a code base can have 100% code coverage yet poor assertions that don't actually provide confidence in code correctness. However, testability is clearly an objectively measurable characteristic. Similarly, teams can measure deployability via a variety of metrics: percentage of successful to failed deployments, how

long deployments take, issues/bugs raised by deployments, and a host of others. Each team bears the responsibility to arrive at a good set of measurements that capture useful data for their organization, both in quality and quantity. Many of these measures come down to team priorities and goals.

Agility and its related parts clearly relate to the software development process. However, that process may impact the structure of the architecture. For example, if ease of deployment and testability are high priorities, then an architect would place more emphasis on good modularity and isolation at the architecture level, an example of an architecture characteristic driving a structural decision. Virtually anything within the scope of a software project may rise to the level of an architecture characteristic if it manages to meet our three criteria, forcing an architect to make design decisions to account for it.

Governance and Fitness Functions

Once architects have established architecture characteristics and prioritized them, how can they make sure that developers will respect those priorities? Modularity is a great example of an aspect of architecture that is important but not urgent; on many software projects, urgency dominates, yet architects still need a mechanism for governance.

Governing Architecture Characteristics

Governance, derived from the Greek word *kubernan* (to steer) is an important responsibility of the architect role. As the name implies, the scope of architecture governance covers any aspect of the software development process that architects (including roles like enterprise architects) want to exert an influence upon. For example, ensuring software quality within an organization falls under the heading of architectural governance because it falls within the scope of architecture, and negligence can lead to disastrous quality problems.

Fortunately, increasingly sophisticated solutions exist to relieve this problem from architects, a good example of the incremental growth in capabilities within the software development ecosystem. The drive toward automation on software projects spawned by Extreme Programming created continuous integration, which led to further automation into operations, which we now call DevOps, continuing through to architectural governance. The book *Building Evolutionary Architectures* (O'Reilly) describes a family of techniques, called fitness functions, used to automate many aspects of architecture governance.

Fitness Functions

The word "evolutionary" in *Building Evolutionary Architectures* comes more from evolutionary computing than biology. One of the authors, Dr. Rebecca Parsons, spent some time in the evolutionary computing space, including tools like genetic algorithms. A genetic algorithm executes and produces an answer and then undergoes mutation by well-known techniques defined within the evolutionary computing world. If a developer tries to design a genetic algorithm to produce some beneficial outcome, they often want to guide the algorithm, providing an objective measure indicating the quality of the outcome. That guidance mechanism is called a *fitness function*: an object function used to assess how close the output comes to achieving the aim. For example, suppose a developer needed to solve the traveling salesperson problem, a famous problem used as a basis for machine learning. Given a salesperson and a list of cities they must visit, with distances between them, what is the optimum route? If a developer designs a genetic algorithm to solve this problem, one fitness function might evaluate the length of the route, as the shortest possible one represents highest success. Another fitness function might be to evaluate the overall cost associated with the route and attempt to keep cost at a minimum. Yet another might be to evaluate the time the traveling salesperson is away and optimize to shorten the total travel time.

Practices in evolutionary architecture borrow this concept to create an *architecture fitness function*:

Architecture fitness function
: Any mechanism that provides an objective integrity assessment of some architecture characteristic or combination of architecture characteristics

Fitness functions are not some new framework for architects to download, but rather a new perspective on many existing tools. Notice in the definition the phrase *any mechanism*—the verification techniques for architecture characteristics are as varied as the characteristics are. Fitness functions overlap many existing verification mechanisms, depending on the way they are used: as metrics, monitors, unit testing libraries, chaos engineering, and so on, illustrated in Figure 6-2.

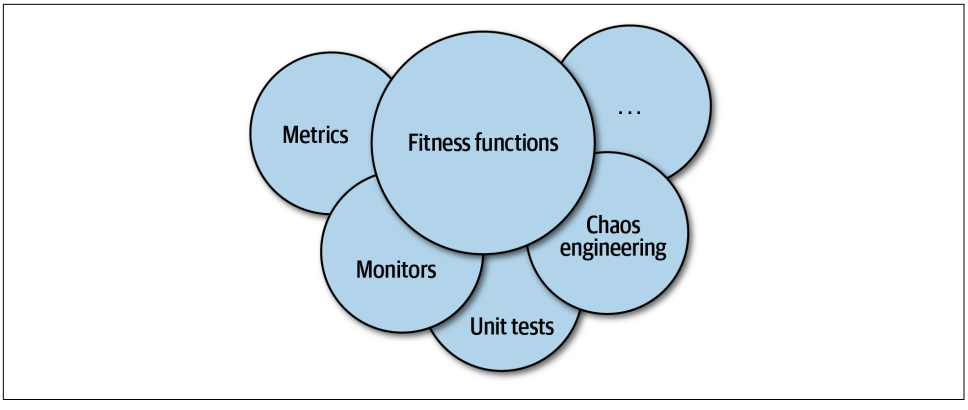

Figure 6-2. The mechanisms of fitness functions

Many different tools may be used to implement fitness functions, depending on the architecture characteristics. For example, in "Coupling" on page 44 we introduced metrics to allow architects to assess modularity. Here are a couple of examples of fitness functions that test various aspects of modularity.

Cyclic dependencies

Modularity is an implicit architecture characteristic that most architects care about, because poorly maintained modularity harms the structure of a code base; thus, architects should place a high priority on maintaining good modularity. However, forces work against the architect's good intentions on many platforms. For example, when coding in any popular Java or .NET development environment, as soon as a developer references a class not already imported, the IDE helpfully presents a dialog asking the developers if they would like to auto-import the reference. This occurs so often that most programmers develop the habit of swatting the auto-import dialog away like a reflex action. However, arbitrarily importing classes or components between one another spells disaster for modularity. For example, Figure 6-3 illustrates a particularly damaging anti-pattern that architects aspire to avoid.

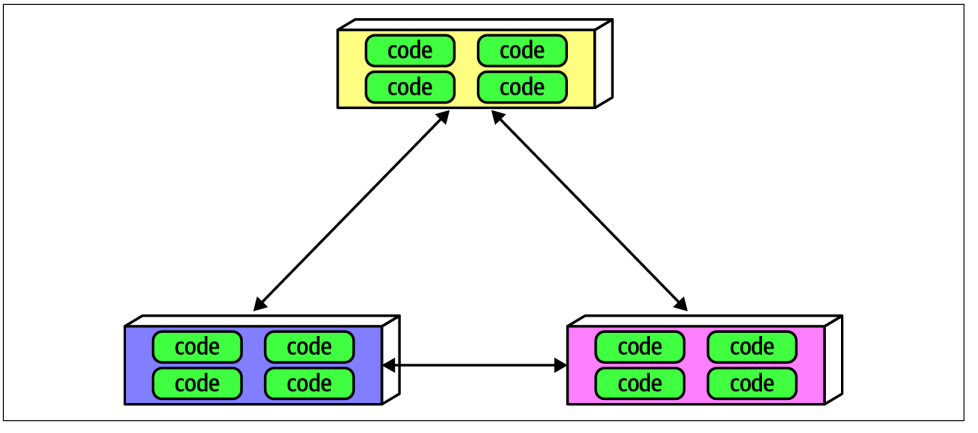

Figure 6-3. Cyclic dependencies between components

In Figure 6-3, each component references something in the others. Having a network of components such as this damages modularity because a developer cannot reuse a single component without also bringing the others along. And, of course, if the other components are coupled to other components, the architecture tends more and more toward the Big Ball of Mud anti-pattern. How can architects govern this behavior without constantly looking over the shoulders of trigger-happy developers? Code reviews help but happen too late in the development cycle to be effective. If an architect allows a development team to rampantly import across the code base for a week until the code review, serious damage has already occurred in the code base.

The solution to this problem is to write a fitness function to look after cycles, as shown in Example 6-2.

Example 6-2. Fitness function to detect component cycles

```
public class CycleTest {
    private JDepend jdepend;

    @BeforeEach
    void init() {
          jdepend = new JDepend();
          jdepend.addDirectory("/path/to/project/persistence/classes");
          jdepend.addDirectory("/path/to/project/web/classes");
          jdepend.addDirectory("/path/to/project/thirdpartyjars");
    }

    @Test
    void testAllPackages() {
          Collection packages = jdepend.analyze();
          assertEquals("Cycles exist", false, jdepend.containsCycles());
```

```
    }
}
```

In the code, an architect uses the metrics tool JDepend to check the dependencies between packages. The tool understands the structure of Java packages and fails the test if any cycles exist. An architect can wire this test into the continuous build on a project and stop worrying about the accidental introduction of cycles by trigger-happy developers. This is a great example of a fitness function guarding the important rather than urgent practices of software development: it's an important concern for architects yet has little impact on day-to-day coding.

Distance from the main sequence fitness function

In "Coupling" on page 44, we introduced the more esoteric metric of distance from the main sequence, which architects can also verify using fitness functions, as shown in Example 6-3.

Example 6-3. Distance from the main sequence fitness function

```
@Test
void AllPackages() {
    double ideal = 0.0;
    double tolerance = 0.5; // project-dependent
    Collection packages = jdepend.analyze();
    Iterator iter = packages.iterator();
    while (iter.hasNext()) {
      JavaPackage p = (JavaPackage)iter.next();
      assertEquals("Distance exceeded: " + p.getName(),
            ideal, p.distance(), tolerance);
    }
}
```

In the code, the architect uses JDepend to establish a threshold for acceptable values, failing the test if a class falls outside the range.

This is both an example of an objective measure for an architecture characteristic and the importance of collaboration between developers and architects when designing and implementing fitness functions. The intent is not for a group of architects to ascend to an ivory tower and develop esoteric fitness functions that developers cannot understand.

Architects must ensure that developers understand the purpose of the fitness function before imposing it on them.

The sophistication of fitness function tools has increased over the last few years, including some special purpose tools. One such tool is ArchUnit, a Java testing framework inspired by and using several parts of the JUnit ecosystem. ArchUnit provides a variety of predefined governance rules codified as unit tests and allows architects to write specific tests that address modularity. Consider the layered architecture illustrated in Figure 6-4.

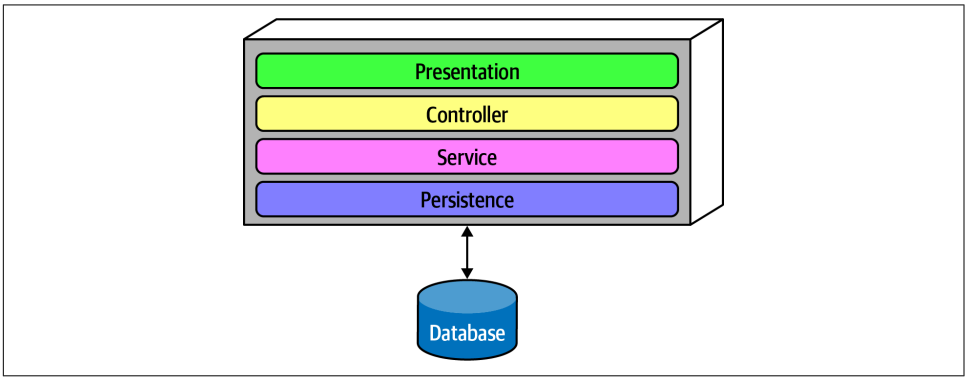

Figure 6-4. Layered architecture

When designing a layered monolith such as the one in Figure 6-4, the architect defines the layers for good reason (motivations, trade-offs, and other aspects of the layered architecture are described in Chapter 10). However, how can the architect ensure that developers will respect those layers? Some developers may not understand the importance of the patterns, while others may adopt a "better to ask forgiveness than permission" attitude because of some overriding local concern such as performance. But allowing implementers to erode the reasons for the architecture hurts the long-term health of the architecture.

ArchUnit allows architects to address this problem via a fitness function, shown in Example 6-4.

Example 6-4. ArchUnit fitness function to govern layers

```
layeredArchitecture()
    .layer("Controller").definedBy("..controller..")
    .layer("Service").definedBy("..service..")
    .layer("Persistence").definedBy("..persistence..")

    .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
    .whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
```

In Example 6-4, the architect defines the desirable relationship between layers and writes a verification fitness function to govern it.

A similar tool in the .NET space, NetArchTest, allows similar tests for that platform; a layer verification in C# appears in Example 6-5.

Example 6-5. NetArchTest for layer dependencies

```
// Classes in the presentation should not directly reference repositories
var result = Types.InCurrentDomain()
    .That()
    .ResideInNamespace("NetArchTest.SampleLibrary.Presentation")
    .ShouldNot()
    .HaveDependencyOn("NetArchTest.SampleLibrary.Data")
    .GetResult()
    .IsSuccessful;
```

Another example of fitness functions is Netflix's Chaos Monkey and the attendant Simian Army. In particular, the *Conformity*, *Security*, and *Janitor* Monkeys exemplify this approach. The Conformity Monkey allows Netflix architects to define governance rules enforced by the monkey in production. For example, if the architects decided that each service should respond usefully to all RESTful verbs, they build that check into the Conformity Monkey. Similarly, the Security Monkey checks each service for well-known security defects, like ports that shouldn't be active and configuration errors. Finally, the Janitor Monkey looks for instances that no other services route to anymore. Netflix has an evolutionary architecture, so developers routinely migrate to newer services, leaving old services running with no collaborators. Because services running on the cloud consume money, the Janitor Monkey looks for orphan services and disintegrates them out of production.

The Origin of the Simian Army

When Netflix decided to move its operations to Amazon's cloud, the architects worried over the fact that they no longer had control over operations—what happens if a defect appears operationally? To solve this problem, they spawned the discipline of Chaos Engineering with the original Chaos Monkey, and eventually the Simian Army. The Chaos Monkey simulated general chaos within the production environment to see how well their system would endure it. Latency was a problem with some AWS instances, thus the Chaos Monkey would simulate high latency (which was such a problem, they eventually created a specialized monkey for it, the Latency Monkey). Tools such as the Chaos Kong, which simulates an entire Amazon data center failure, helped Netflix avoid such outages when they occured for real.

Chaos engineering offers an interesting new perspective on architecture: it's not a question of if something will eventually break, but when. Anticipating those breakages and tests to prevent them makes systems much more robust.

A few years ago, the influential book *The Checklist Manifesto* by Atul Gawande (Picador) described how professions such as airline pilots and surgeons use checklists (sometimes legally mandated). It's not because those professionals don't know their jobs or are forgetful. Rather, when professionals do a highly detailed job over and over, it becomes easy for details to slip by; a succinct checklist forms an effective reminder. This is the correct perspective on fitness functions—rather than a heavyweight governance mechanism, fitness functions provide a mechanism for architects to express important architectural principles and automatically verify them. Developers know that they shouldn't release insecure code, but that priority competes with dozens or hundreds of other priorities for busy developers. Tools like the Security Monkey specifically, and fitness functions generally, allow architects to codify important governance checks into the substrate of the architecture.